

SMART TRAFFIC SIGNAL SYSTEM USING YOLOv8

An AI-Based Computer Vision System for
Car Detection and Traffic Signal Decision

Project Title	Smart Traffic Signal System using YOLOv8
Author	AJIT CHOWDHURY
Course	B.Sc. Computer Science
Institution	Mankar College
University	Burdwan University
Department	Computer Science
Project Domain	Artificial Intelligence and Computer Vision
Programming Language	Python

Project Report

Table of Contents

Section	Page
1. Title and Overview	3
2. Introduction	4
3. Literature Review	5
4. System Design and Methodology	7
5. Implementation	10
6. Results and Testing	13
7. Conclusion and Future Work	16
Final Project Summary	18
Technology Stack	19
References	20

Note: Page numbers are an approximate guide because the final layout is generated automatically. The report also includes an architecture diagram, a system flowchart, project structure, and actual application screenshots.

1. Title And Overview

1.1 Project Title

Smart Traffic Signal System using YOLOv8

1.2 Author Details

Name: AJIT CHOWDHURY

Course: B.Sc. Computer Science

Institution: Mankar College

University: Burdwan University

Project Area: Artificial Intelligence, Deep Learning and Computer Vision

1.3 Abstract

Traffic congestion is a major problem in urban areas, particularly at road intersections where the number of vehicles can change continuously. Conventional traffic signal systems generally operate according to predefined timing schedules and may not respond efficiently to variations in traffic density. This project presents a prototype of a **Smart Traffic Signal System using YOLOv8**, a deep-learning-based object detection model, to detect and count cars from traffic video.

The system is implemented using **Python, OpenCV, Ultralytics YOLOv8, and Streamlit**. A traffic video is uploaded through a Streamlit web interface and processed frame by frame. YOLOv8n, using its pretrained weights, identifies cars in each frame. The detected car count is then used by a rule-based traffic signal mechanism to determine whether the signal should be GREEN, YELLOW, or RED. In the current prototype, 0-5 detected cars result in a GREEN signal, 6-10 result in a YELLOW signal, and more than 10 result in a RED signal.

The project demonstrates how deep-learning-based object detection can be combined with a simple decision-making mechanism to create an intelligent traffic-management prototype. The system provides visual bounding boxes, vehicle counts, and traffic-signal status through a web-based interface.

Keywords: Artificial Intelligence, Computer Vision, YOLOv8, Object Detection, OpenCV, Streamlit, Traffic Signal, Vehicle Detection, Deep Learning.

2. Introduction

2.1 Background

The rapid growth of urban populations and the increasing number of vehicles on roads have resulted in significant traffic congestion. Traffic congestion causes longer travel times, increased fuel consumption, environmental pollution, and inefficient use of road infrastructure.

Traditional traffic signals commonly use fixed-time schedules. For example, a traffic light may remain GREEN for a predefined number of seconds regardless of whether there are many vehicles waiting or almost no vehicles

present. Such systems do not continuously understand the actual traffic conditions.

Artificial Intelligence and Computer Vision provide an opportunity to develop more responsive traffic-management systems. A camera can capture road traffic, while a computer-vision model can analyze the captured video and identify vehicles. The number of detected vehicles can then be used to estimate traffic density and make a signal decision.

In this project, **YOLOv8** is used as the primary object-detection technology. YOLO, which stands for "You Only Look Once", is designed to perform object detection efficiently by identifying objects and their locations within images or video frames.

The project focuses specifically on **car detection**. The detected number of cars is used as the primary input for a prototype traffic-signal decision system.

2.2 Problem Statement

Traditional fixed-time traffic signals do not automatically respond to changes in traffic density. A road with a large number of waiting vehicles may receive the same signal duration as a road with very few vehicles.

The problem addressed by this project is therefore:

How can computer vision and deep-learning-based object detection be used to detect cars in traffic video and make a basic traffic-signal decision according to the detected traffic density?

The proposed system attempts to address this problem by automatically detecting cars from traffic video and using the detected car count to determine a traffic signal state.

2.3 Objectives

The major objectives of the project are:

1. To develop a computer-vision-based traffic monitoring system.
2. To detect cars from traffic video using YOLOv8.
3. To process traffic video frame by frame.
4. To count the number of detected cars.
5. To use the car count as an indicator of traffic density.
6. To automatically determine a GREEN, YELLOW, or RED traffic signal.
7. To display detected cars using bounding boxes.
8. To display the current car count to the user.
9. To provide a simple web interface using Streamlit.
10. To develop a practical AI and Computer Vision project suitable for further expansion.
11. To maintain and publish the project source code using Git and GitHub.

3. Literature Review

3.1 Traditional Traffic Signal Systems

Traditional traffic signal systems commonly rely on fixed timing or manually configured schedules. These systems can work effectively under predictable traffic conditions, but their response is limited when traffic density changes significantly.

For example, a signal may continue providing GREEN time to a lane even when only a small number of vehicles are present, while another lane may have a larger queue.

This limitation motivates the use of automated traffic-monitoring techniques.

3.2 Computer Vision for Traffic Monitoring

Computer Vision enables computers to analyze images and videos and identify meaningful objects. In traffic-management applications, cameras can be used to capture road scenes while computer-vision algorithms identify vehicles.

A traffic-monitoring system can use information such as:

- Number of vehicles
- Vehicle locations
- Traffic density
- Vehicle movement
- Lane occupancy

This information can be used to assist automated traffic-management decisions.

3.3 Haar Cascade Classifier

During the earlier development of this project, a **Haar Cascade Classifier** approach was explored for vehicle detection.

OpenCV provides Haar Cascade functionality for object detection using trained cascade classifiers. In the prototype, a car cascade file named `cars.xml` was used.

The initial system attempted to detect cars using OpenCV's `detectMultiScale()` functionality. The detected vehicles were then counted and used for traffic-signal decisions.

However, the Haar Cascade approach produced false detections in some traffic scenes. Objects such as road signs, bushes, and other visual patterns could sometimes be interpreted as vehicles.

This limitation led to the adoption of a deep-learning-based object-detection approach.

3.4 YOLO-Based Object Detection

YOLO is a deep-learning-based object-detection approach designed to detect objects within images efficiently. Instead of treating object detection as multiple separate stages, YOLO performs object detection in a unified framework.

For this project, **YOLOv8n** was selected because it provides a lightweight pretrained model suitable for experimentation and real-time or near-real-time computer-vision applications.

The project uses the pretrained model:

```
model = YOLO("yolov8n.pt")
```

The project does **not** train YOLOv8 from scratch. Instead, the pretrained YOLOv8n model is used to detect cars.

3.5 Selection of YOLOv8

YOLOv8 was selected because:

- It provides pretrained object-detection models.
- It can process images and video frames.
- It provides bounding-box detections.
- It supports multiple object classes.
- The YOLOv8n model is relatively lightweight.
- It can be integrated directly with Python.
- The Ultralytics library simplifies model loading and inference.

For the current project, only the **car class** is selected.

The COCO class identifier for a car is:

```
vehicle_classes = [2]
```

Therefore, the project intentionally focuses on car detection rather than detecting every type of road vehicle.

4. System Design And Methodology

4.1 System Overview

The proposed system receives a traffic video from the user through a Streamlit interface. The video is temporarily stored and opened using OpenCV. Each frame is passed to the YOLOv8 model.

YOLOv8 analyzes the frame and returns detected objects along with their class information and bounding-box coordinates.

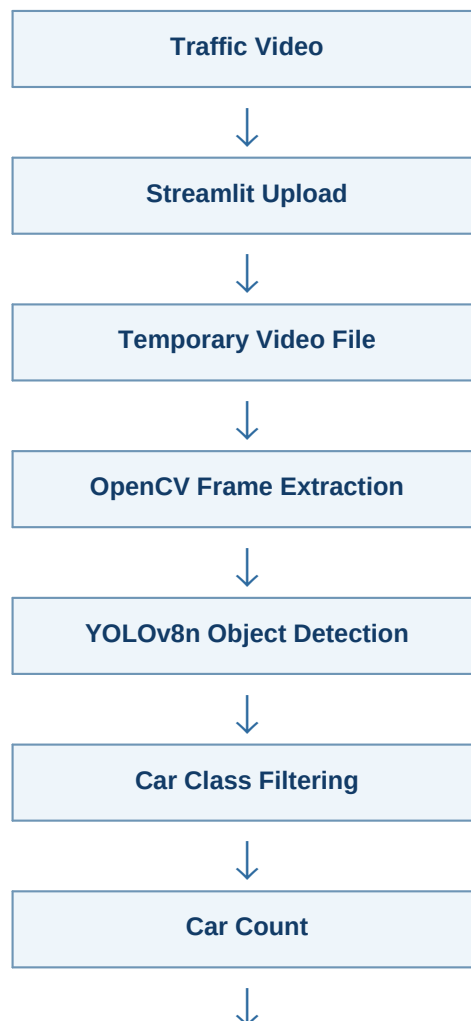
The system checks whether each detected object belongs to the car class. If it does, the object is counted as a car.

The total number of detected cars is then used by the traffic-signal logic.

The overall workflow is:

Component	Role
Traffic Video	Input traffic video supplied by the user
Streamlit Upload	Web interface for selecting the video
Temporary File	Uploaded video is stored temporarily
OpenCV	Reads and processes frames
YOLOv8n	Detects objects in each frame
Car Filtering	Keeps only the car class (COCO ID 2)
Car Count	Counts detected cars in the current frame
Signal Logic	Maps the count to GREEN/YELLOW/RED
Streamlit Output	Displays detections and signal status

System Flowchart



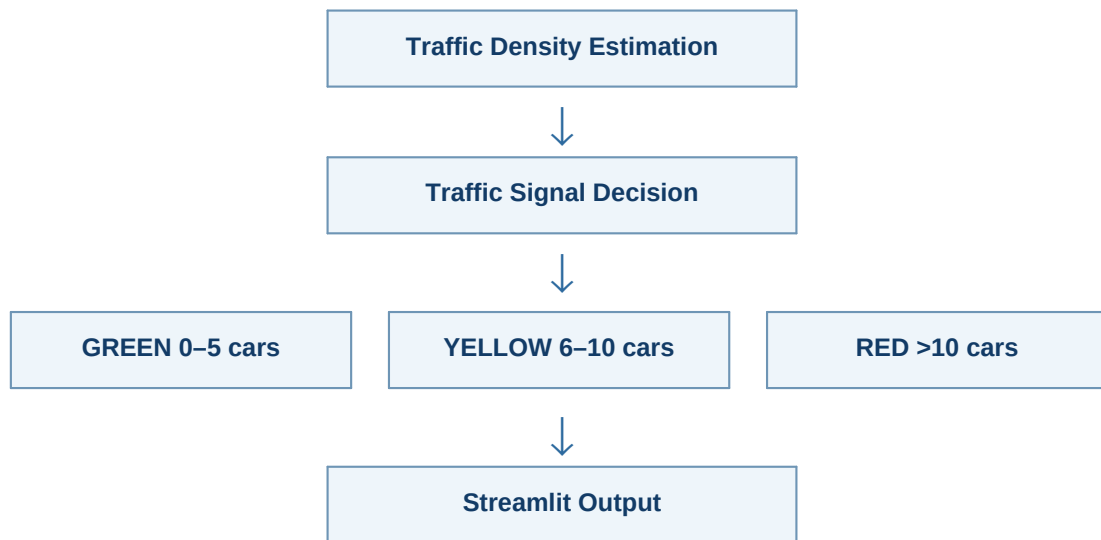


Figure: End-to-end processing flow used by the implemented YOLOv8 traffic-signal prototype.

```

Traffic Video
|
↓
Streamlit Upload
|
↓
Temporary File
|
↓
OpenCV Video
Processing
|
↓
YOLOv8n
|
↓
Object Detection
|
↓
Car Filtering
|
↓
Car Count
|
↓
Traffic Density
|
↓
Traffic Signal Logic
|
████████████████████████████████████████
↓ ↓ ↓
GREEN YELLOW RED
| | |
████████████████████████████████████████
↓
Streamlit Display
  
```

4.2 Data Collection

The project uses traffic video as the input data.

The application allows the user to upload video files with supported formats such as:

- MP4
- AVI
- MOV

The system does not require a manually labeled dataset for the current implementation because it uses the pretrained YOLOv8n model.

The uploaded video acts as the input source for testing the traffic-detection system.

4.3 Video Processing

After the user uploads a traffic video, the application creates a temporary file using Python's `tempfile` module.

The video is then opened using OpenCV:

```
cap = cv2.VideoCapture(tfile.name)
```

The application reads the video frame by frame:

```
ret, frame = cap.read()
```

Processing continues until the video reaches its end.

4.4 YOLOv8 Model

The project uses the Ultralytics implementation of YOLOv8.

The model is loaded using:

```
from ultralytics import YOLO  
model = YOLO("yolov8n.pt")
```

The `yolov8n.pt` file represents the pretrained YOLOv8 Nano model.

The model receives an individual video frame:

```
results = model(frame)
```

The resulting detections contain information about detected objects, including their classes and bounding boxes.

4.5 Car Detection

YOLOv8 can identify multiple classes of objects. However, this project is intentionally designed to detect cars only.

The selected class is:

```
vehicle_classes = [2]
```

Here, class ID 2 corresponds to the car class in the model's COCO-based class mapping.

For each detected object, the class ID is extracted:

```
cls = int(box.cls[0])
```

The system then checks:

```
if cls in vehicle_classes:
```

If the condition is true, the object is treated as a detected car.

4.6 Bounding Boxes

For every detected car, the model provides bounding-box coordinates.

The coordinates are extracted using:

```
x1, y1, x2, y2 = map(int, box.xyxy[0])
```

The system then stores the coordinates and draws a rectangle around the detected car:

```
cv2.rectangle(  
    frame,  
    (x1, y1),  
    (x2, y2),  
    (0, 255, 0),  
    2  
)
```

This allows the user to visually verify which objects have been detected as cars.

4.7 Vehicle Counting

For each frame, a counter is initialized:

```
count = 0
```

Whenever a detected object is identified as a car, the counter is increased:

```
count += 1
```

The resulting value represents the number of cars detected in that particular frame.

The current implementation therefore performs **frame-level car counting** rather than persistent vehicle tracking across multiple frames.

4.8 Traffic Signal Logic

The car count is used as a simple traffic-density indicator.

The current signal logic is:

```
if count <= 5:  
    signal = "GREEN"
```

```
elif count <= 10:
    signal = "YELLOW"

else:
    signal = "RED"
```

Therefore:

Detected Cars	Signal
-----;	:-----
0-5	GREEN
6-10	YELLOW
More than 10	RED

This rule-based logic is a prototype for demonstrating how AI-based vehicle detection can be connected to traffic-signal decision making.

The thresholds are currently predefined and are not learned automatically from traffic data.

4.9 Traffic Light Visualization

The application also generates a visual traffic-light representation using OpenCV.

Three circles are drawn:

- Red
- Yellow
- Green

The active signal is highlighted using a brighter color.

This provides an easily understandable visual representation of the system's decision.

4.10 Streamlit Interface

The application uses Streamlit to create a simple web interface.

The user is presented with a file uploader:

```
uploaded_file = st.file_uploader(
    "Upload a traffic video",
    type=["mp4", "avi", "mov"]
)
```

After uploading a video, the application processes it and displays the results directly in the browser.

The processed frames are displayed using Streamlit:

```
stframe.image(  
    frame,  
    channels="RGB",  
    use_container_width=True  
)
```

5. Implementation

5.1 Programming Language

The entire project is implemented using **Python**.

Python was selected because it provides extensive support for:

- Artificial Intelligence
- Machine Learning
- Deep Learning
- Computer Vision
- Video processing
- Web application development

5.2 Tools and Technologies

The major tools and technologies used during development are:

Python

Python is the primary programming language used to implement the project.

OpenCV

OpenCV is used for:

- Reading video
- Processing frames
- Drawing bounding boxes
- Drawing traffic-light indicators
- Displaying visual information
- Converting image color formats

Ultralytics YOLOv8

The Ultralytics library provides the YOLOv8 implementation used for car detection.

The project uses:

```
from ultralytics import YOLO
```

and:

```
YOLO("yolov8n.pt")
```

Streamlit

Streamlit is used to create the web-based user interface.

It provides:

- Video file upload
- Application title
- Video display
- Real-time processing output
- Browser-based interaction

NumPy / Python Numerical Environment

The Python environment used for Computer Vision development supports numerical image and frame processing through the libraries required by OpenCV and the YOLO implementation.

Jupyter Notebook

Jupyter Notebook was used during the development and experimentation stage.

The project contains notebooks such as:

```
Traffic.ipynb  
Traffic2.ipynb
```

These were used for experimentation and Computer Vision development.

Temporary File Handling

Python's built-in `tempfile` module is used to temporarily store uploaded videos before OpenCV processes them.

Git

Git is used for:

- Version control
- Tracking project changes
- Creating commits
- Managing the project repository

GitHub

GitHub is used to publish and maintain the source code online.

The project repository contains the source code, notebooks, README documentation, and supporting project files.

5.3 Development Environment

The project was developed on a Windows computer using the command-line environment and Visual Studio Code/terminal-based workflow.

Git was installed using Windows Package Manager:

```
winget install --id Git.Git -e --source winget
```

The Git repository was initialized and the project was uploaded to GitHub.

5.4 Project Structure

The current project structure is:

```
Smart-Traffic-Signal-System/  
■  
■■■ README.md  
■■■ app_cv.py  
■■■ cars.xml  
■■■ Traffic.ipynb  
■  
■■■ Deep Learning Relate/  
■■■ app_DL.py  
■■■ Traffic2.ipynb
```

app_cv.py

This file represents the Computer Vision application developed during the earlier stage of the project.

cars.xml

This is the Haar Cascade classifier file used during the earlier vehicle-detection approach.

Traffic.ipynb

This notebook contains Computer Vision experimentation and development work.

Deep Learning Relate/Traffic2.ipynb

This notebook contains the deep-learning-related experimentation.

Deep Learning Relate/app_DL.py

This is the main YOLOv8-based Streamlit application.

README.md

The README provides an overview of the project, technologies, workflow, installation and usage information.

5.5 Main YOLO Application

The main application performs the following steps:

1. Start Streamlit.
2. Display the application title.
3. Allow the user to upload a traffic video.
4. Save the uploaded video temporarily.
5. Load YOLOv8n.
6. Open the video using OpenCV.
7. Read each frame.
8. Run YOLOv8 detection.
9. Filter detections for the car class.
10. Count detected cars.
11. Determine the signal.
12. Draw bounding boxes.
13. Display the car count.
14. Display the traffic signal.
15. Display the processed frame.
16. Continue until the video ends.

5.6 Model Training

The current implementation uses the **pretrained YOLOv8n model**.

Therefore, the project does not include custom YOLO model training, dataset annotation, or custom hyperparameter optimization.

The model is used directly for inference:

```
model = YOLO("yolov8n.pt")
```

This approach allows the project to demonstrate practical application of an existing deep-learning object detector without requiring a large custom training dataset.

5.7 Model Configuration

The project intentionally selects only the car class:

```
vehicle_classes = [2]
```

This makes the current system a **car-focused traffic monitoring prototype**.

The system could be extended later to include other vehicle categories.

6. Results And Testing

6.1 System Output

The developed system successfully provides a visual interface for processing traffic video.

The output contains:

- Traffic video frames
- Car bounding boxes
- Detected car count
- Traffic signal status
- Graphical traffic-light indicator

The user can upload a supported traffic video and observe the model's detections as the video is processed.

6.2 Car Detection

When a car is detected, YOLOv8 generates a bounding box around the vehicle.

The bounding box allows the user to visually confirm the model's detection.

The number of detected cars is displayed on the video frame using OpenCV text rendering.

Example output concept:

```
Cars: 4  
Signal: GREEN
```

If the detected count increases:

```
Cars: 8  
Signal: YELLOW
```

For higher traffic density:

```
Cars: 13  
Signal: RED
```

6.3 Traffic Signal Testing

The signal mechanism was tested using the predefined thresholds.

Test Case 1: Low Traffic

Detected cars: 5

Expected result:

GREEN

The system identifies the traffic level as low according to the prototype threshold.

Test Case 2: Medium Traffic

Detected cars: 8

Expected result:

YELLOW

The system identifies the traffic level as medium.

Test Case 3: High Traffic

Detected cars: 12

Expected result:

RED

The system identifies the traffic level as high.

6.4 Functional Testing

The major functional components were tested:

Component	Test Result
Streamlit application launch	Successful
Video upload	Successful
Video reading using OpenCV	Successful
YOLOv8 model loading	Successful
Car detection	Successful
Bounding-box display	Successful
Car counting	Successful
Signal decision	Successful
Traffic-light visualization	Successful
GitHub project upload	Successful

6.5 User Interface

The Streamlit interface provides a simple interaction model.

The user does not need to interact with command-line computer-vision operations after starting the application. Instead, the user can upload a traffic video through the browser interface.

The processed video is displayed in the application window.

Recommended report screenshot:

A screenshot of the Streamlit application showing:

- YOLO-detected cars
- Bounding boxes
- Car count
- Signal status
- Traffic-light visualization

6.6 Limitations Observed

Although the prototype successfully demonstrates the concept, several limitations remain.

Frame-Level Counting

The current system counts detections independently in each frame. It does not track individual vehicles across frames.

Therefore, the count represents detected cars in a frame rather than a persistent count of unique vehicles.

Fixed Signal Thresholds

The thresholds:

```
0-5
6-10
11+
```

are manually defined.

They are not dynamically learned from historical traffic data.

Camera Perspective

Detection performance can depend on:

- Camera angle
- Lighting

- Video quality
- Vehicle size
- Occlusion
- Weather conditions

Pretrained Model

The system uses a pretrained YOLOv8n model rather than a model trained specifically on a custom local traffic dataset.

Prototype Traffic Control

The traffic-light output is a software simulation. It is not connected to a physical traffic signal controller or an actual road intersection.

Therefore, the project should be considered a **Computer Vision prototype for intelligent traffic-signal decision making**, rather than a production traffic-control system.

7. Conclusion And Future Work

7.1 Conclusion

The **Smart Traffic Signal System using YOLOv8** successfully demonstrates the integration of Artificial Intelligence, Computer Vision, video processing, and a web-based interface into a single application.

The project progressed from an initial Haar Cascade-based vehicle-detection approach to a deep-learning-based YOLOv8 implementation. The Haar Cascade approach provided an initial understanding of traditional computer-vision object detection but produced unwanted detections in some scenes. YOLOv8 was therefore selected for the main implementation.

The final system uses the pretrained YOLOv8n model to detect cars from traffic video. OpenCV handles video processing and visualization, while Streamlit provides an interactive web interface. The number of detected cars is used to determine a prototype traffic-signal state.

The project demonstrates practical knowledge of:

- Artificial Intelligence
- Deep Learning
- Object Detection
- Computer Vision
- Video Processing
- Python programming
- Streamlit application development
- Git version control

- GitHub project management

The project also provides a foundation for developing a more advanced intelligent traffic-management system.

7.2 Future Work

Several improvements can be implemented in future versions.

1. Vehicle Tracking

An object-tracking algorithm can be integrated to track individual cars across consecutive frames.

This would reduce the limitations of frame-by-frame counting.

2. Adaptive Signal Timing

Instead of using fixed thresholds, the system could calculate traffic density dynamically and determine signal duration based on actual traffic conditions.

3. Multiple Vehicle Classes

The current implementation detects cars only.

Future versions could detect:

- Cars
- Motorcycles
- Buses
- Trucks

4. Real-Time Camera Input

Instead of uploading a recorded video, the system could process a live CCTV or webcam feed.

5. Multi-Lane Analysis

The system could identify separate lanes and calculate the traffic density for each lane independently.

6. Custom YOLO Training

A custom traffic dataset could be collected and annotated to train or fine-tune a YOLO model specifically for local traffic conditions.

7. Emergency Vehicle Detection

The system could be extended to recognize emergency vehicles and provide priority to their lane.

8. Traffic History and Analytics

The application could store traffic counts over time and provide:

- Traffic-density graphs
- Peak-hour analysis
- Daily traffic statistics
- Historical comparisons

9. Physical Traffic Signal Integration

A future version could connect the AI system to a hardware-based traffic-light controller in a controlled experimental environment.

10. Cloud Deployment

The application could eventually be deployed online so that authorized users can access the system remotely.

Final Project Summary

The project combines **YOLOv8**, **OpenCV**, **Python**, **Streamlit**, and **Git/GitHub** to demonstrate an AI-based approach to traffic monitoring.

The final system follows this pipeline:

```
Traffic Video
↓
OpenCV
↓
YOLOv8n
↓
Car Detection
↓
Car Counting
↓
Traffic Density Estimation
↓
Signal Decision
↓
GREEN / YELLOW / RED
↓
Streamlit Visualization
```

The project serves as a practical demonstration of how a pretrained deep-learning model can be integrated into a complete Computer Vision application and connected to a decision-making mechanism.

TECHNOLOGY STACK

Category	Technology
Programming Language	Python
AI / Deep Learning	YOLOv8
Model Framework	Ultralytics

Category	Technology
Computer Vision	OpenCV
Web Interface	Streamlit
Video Processing	OpenCV
Development / Experimentation	Jupyter Notebook
Earlier Detection Approach	Haar Cascade
Version Control	Git
Code Hosting	GitHub
Input	Traffic Video
Primary Detection Target	Cars
Model	YOLOv8n
Output	Car Count + Traffic Signal

PROJECT REPOSITORY

The source code and project documentation are maintained in the GitHub repository:

Smart Traffic Signal System using YOLOv8

<https://github.com/AJITC600/Smart-Traffic-Signal-System>

Visual Results and Implementation Evidence

The following figures are taken from the implemented Streamlit application and show the YOLOv8 car detections, the frame-level car count, and the simulated traffic-signal output.



Figure 1. Streamlit application showing YOLOv8 car detections, a car count of 3, and GREEN signal output.

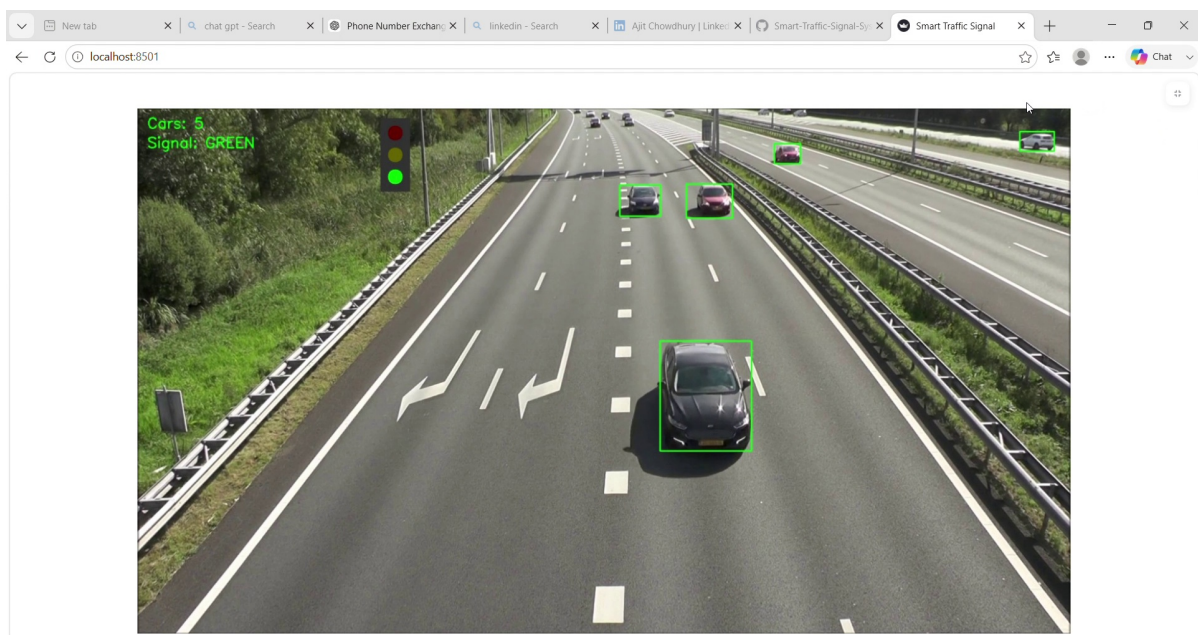


Figure 2. YOLOv8 detection output with five detected cars and the corresponding GREEN signal.

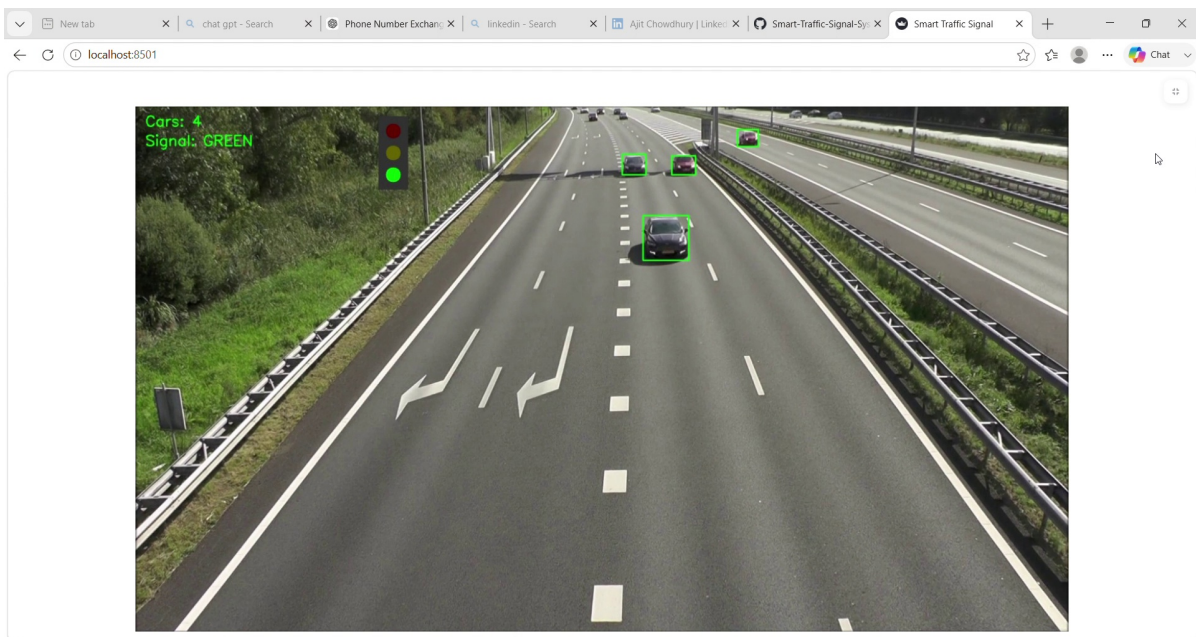


Figure 3. YOLOv8 detection output with four detected cars and the corresponding GREEN signal.

Project Structure and System Architecture

Project Folder Structure

```
Smart-Traffic-Signal-System/  
├──  
│   ├── README.md  
│   ├── app_cv.py  
│   ├── cars.xml  
│   ├── Traffic.ipynb  
│   └──  
│       ├── Deep Learning Relate/  
│       ├── app_DL.py  
│       └── Traffic2.ipynb
```

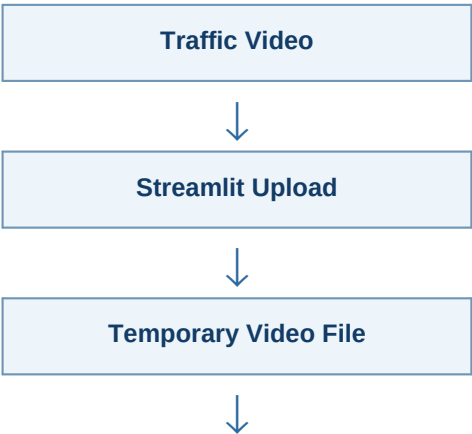
Figure: Project structure containing the earlier Haar Cascade implementation, the YOLOv8 Streamlit application, notebooks, and repository documentation.

The implementation is organized so that the earlier OpenCV/Haar Cascade work remains available for comparison, while the final deep-learning application is located in the **Deep Learning Relate** folder.

Architecture Summary

Component	Role
Traffic Video	Input traffic video supplied by the user
Streamlit Upload	Web interface for selecting the video
Temporary File	Uploaded video is stored temporarily
OpenCV	Reads and processes frames
YOLOv8n	Detects objects in each frame
Car Filtering	Keeps only the car class (COCO ID 2)
Car Count	Counts detected cars in the current frame
Signal Logic	Maps the count to GREEN/YELLOW/RED
Streamlit Output	Displays detections and signal status

System Flowchart



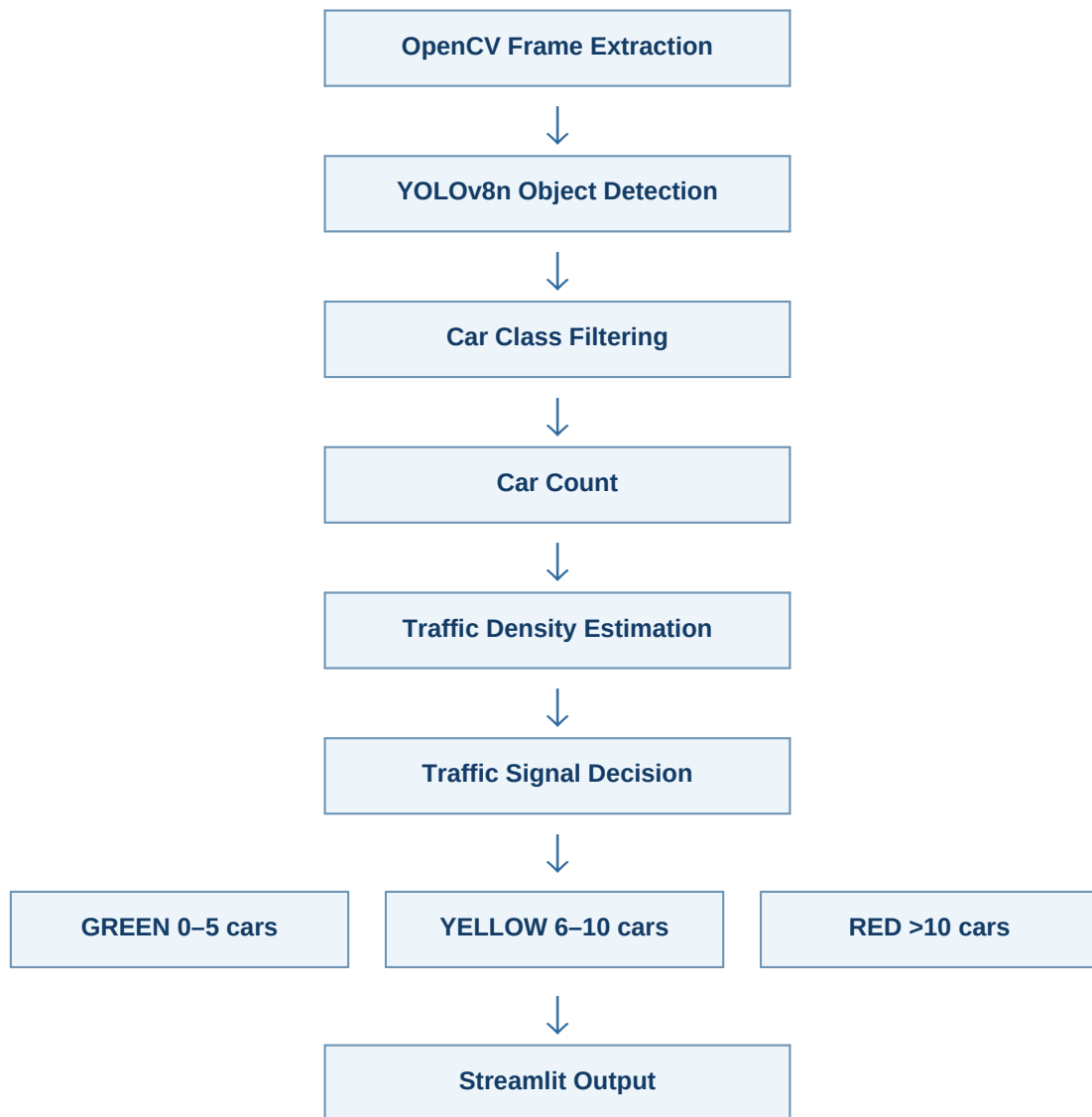


Figure: End-to-end processing flow used by the implemented YOLOv8 traffic-signal prototype.

References

Source	Description	Reference
Ultralytics	YOLO / Ultralytics documentation	https://docs.ultralytics.com/
OpenCV	Open Source Computer Vision Library	https://opencv.org/
Streamlit	Streamlit documentation and application framework	https://streamlit.io/
Git	Git version control system documentation	https://git-scm.com/doc
GitHub	Project repository - Smart Traffic Signal System	https://github.com/AJITC600/Smart-Traffic-Signal-System

The technical description, implementation details, thresholds, limitations, and project structure in this report are based on the supplied Report.txt and the implemented project evidence.